PowerDAG：用于自动化配电网分析的可靠智能体AI系统

Emmanuel O. Badmus, Amritanshu Pandey

摘要 — 本文介绍了*PowerDAG*，一个用于自动化复杂配电网分析的智能体*AI*系统。我们通过引入两种创新的主动机制来解决最先进的智能体系统在自动化复杂工程工作流中的可靠性挑战：*(i)* 自适应检索，它使用相似度衰减断点算法动态选择最相关的带注释示例作为上下文；*(ii)* 即时监督，它在执行期间主动拦截并纠正工具使用违规行为。在一个未见过的配电网分析查询基准上，*PowerDAG*在*GPT-5.2*上达到了*100%*的成功率，在较小的开源模型上达到了*94.4%-96.7%*的成功率，显著优于基础*ReAct (41-88%)*、*LangChain (30-90%)* 和 *CrewAI (9-41%)* 基线，超出幅度达*6-50*个百分点。

关键词 — 智能体AI系统，配电网分析，长提示上下文，相关上下文，工作流编排

I. 引言

分布式能源资源（DER）的快速增长，包括屋顶太阳能光伏、电池储能和电动汽车，使得配电网分析变得频繁且复杂[1]。这些分析通过仿真工具（例如，用于电压分布[2]、[3]的潮流）、优化工具（例如，用于量化太阳能削减的动态承载容量[4]）和数据分析[5]来支持运行和规划。每项分析都需要多步骤执行：模型组装、工具编排与执行、结果验证。许多需要这些分析的利益相关者，特别是较小的公用事业公司、监管机构和政策团体，缺乏大规模进行这些分析的工程能力，从而延迟了并网审批并限制了电网现代化[6]。

核心挑战是在成熟的工具（如GridLabD [7]）和数据管道（如高级计量基础设施数据库）上执行多步骤工具序列，同时保持正确的工具选择、参数绑定和环境状态[8]。虽然基于规则的引擎可以自动化标准化的研究模板，但它们不灵活，需要持续的人工维护，并且无法稳健地适应真实世界分析的多样性[9]。LLM提供了一种有前途的替代方案，使用户能够用自然语言描述所需的分析，并接收相应的工具调用编排[10]。

早期方法向LLM提供工具描述，并提示其为工程师编排工具调用序列[11]、[12]，但这仍然需要手动执行和调试[13]。智能体系统通过基于不断演变的交互历史（用户查询、工具描述、先前步骤的动作和观察）提出工具和参数，消除了人工干预。然而，由于在预训练期间对特定领域的过程性知识接触有限，LLM难以生成针对专业领域的有效执行计划[11]。两种主要策略解决了这一问题：监督微调使用特定领域的训练数据调整模型权重[12]，而长上下文学习在提示中提供与任务相关的内容，无需更新权重[14]、[15]。SFT能有效地内化领域模式，但需要大量的计算资源、特定领域的训练数据，并且在工具或数据管道演变时需要完全重新训练。

ICL通过在提示中增加与任务相关的上下文来避免这些成本[14]。然而，ICL的有效性在很大程度上取决于所提供上下文的质量和结构[15]。通用的指导来源（论文、手册）描述概念，但很少指定工具的执行逻辑和推理，因为每个来源通常只涵盖一个工具或方法而忽略管道的其余部分。因此，没有单一来源指定跨工具交互、数据交接或状态更新。例如，存在单独的GridLAB-D和AMI数据库指南，但两者都没有定义将AMI数据映射到一致馈线负载的跨工具编排。

为了弥补这些差距，最近的工作[16]、[17]通过整理专家运行复杂分析的经验并将其整理成带注释的示例，展示了有希望的结果。这些示例以结构化的方式编码了可操作的程序性知识，包括工具调用的编排、状态依赖关系和参数结构。然而，在这种范式中，选择带注释的示例至关重要，因为LLM的上下文

窗口有限，这限制了我们可以包含在提示中的示例数量。此外，不相关的带注释示例会引入分散注意力的工具轨迹，稀释相关的程序性信号，在长上下文中降低工具选择质量，同时增加提示长度和令牌成本[18]。为了弥补这些差距，最近的工作[16]使用基于RAG的相似性检索来为每个查询选择top-k示例。

问题：[16]中的静态top-k检索有两个主要缺点。首先，k的选择引入了固有的权衡：较小的k会遗漏复杂分析查询的潜在工具依赖关系，而较大的k会包含不相关的示例，从而降低工具选择质量并增加上下文大小[18]。其次，在目前的形式中，[16]仅基于查询相似性来隔离相关的带注释示例。虽然这会在当前查询和所选示例之间产生语义匹配，但它可能丢弃具有相关工具依赖关系和编排的示例。

另一个关键挑战出现在工具执行期间。对于未见的查询，智能体会调用LLM，根据相关的带注释示例推荐工具调用。智能体执行这些工具调用，并使用返回消息作为正确性的主要信号。这种方法在有状态环境中很脆弱，因为工具通常会在过时或未初始化的对象上成功执行而不引发异常。例如，如果一个智能体运行潮流求解器但跳过了先决条件的状态更新调用，那么后续的绘图函数将被执行但会产生错误的图。智能体将此静默失败误解为成功并继续执行。为了防止这种不可逆的状态损坏和静默失败，一个主动的监督者必须强制执行依赖约束。

提出的解决方案：为了弥补这些根本性差距，我们在我们的智能体系统PowerDAG中实施了一个双管齐下的策略：

- 自适应检索：我们引入了一种相似度衰减断点算法，通过查询文本和工具调用轨迹来选择带注释示例的子集，以隔离与程序相关的上下文。
- 即时监督：我们引入了一个调用前防护栏，用于验证状态依赖关系，通过阻止无效调用并返回一次性纠正建议来防止状态损坏。

II. 相关工作

A. 用于工具编排的LLM

商业LLM聊天界面（例如ChatGPT、Claude、Gemini）将用户查询与历史对话上下文连接起来，并将其路由到其后端LLM（例如GPT-5.2、Claude Opus 4.5、Gemini 3 Pro）以生成响应。在标准聊天界面中，底层的LLM无法运行外部配电系统仿真器（例如，ChatGPT本身无法运行OpenDSS）。因此，它返回用户可以在界面外执行和验证的指导或代码。电力系统应用包括OpenDSS文件生成[19]、迭代脚本优化[20]和结果可视化[21]。另一种方法是在提示中增加工具描述，并请求带有绑定参数的完整工具调用序列[11]、[12]。这些需要手动工具调用。作为改进，智能体框架采用迭代执行循环，其中LLM提出单个工具调用，观察结果，并根据反馈决定后续步骤[10]、[13]。智能体电网应用包括用于OPF自动化的GridMind [22]、用于 contingency 分析的GridAgent [23]、用于求解器驱动规划的RePower [24]以及用于跨电力系统仿真器的可执行多步骤工作流的X-GridAgent [25]。然而，这些系统依赖于求解器在环检查，但并未隐式强制执行先决条件和跨工具状态依赖关系，从而限制了在有状态管道中的可靠性。

B. 基于示例的条件化编排

智能体框架自动化了工具编排，但需要过程性指导来选择工具、排序调用和满足先决条件依赖关系。因此，最近的系统基于检索到的成功工具使用示例来决策，而不是仅依赖自然语言手册[15]、[26]。工具检索方法还使用来自LLM调用的迭代反馈来优化要使用的工具和示例[27]。一些智能体系统将先前的轨迹存储并检索为过程性记忆，以改善长期任务中的未来决策[28]。其他工作从交互中构建可重用的多步骤轨迹，并通过跨任务的检索和优化来重用它们[29]、[30]。GeoFlow明确地将这些轨迹命名为工作流，并将其表示为带有步骤级工具目标的顶点活动图，从而提高了任务成功率并减少了令牌使用[17]。[16]使配电网分析智能体以经过验证的执行轨迹的带注释示例为条件，并报告在其实验中Pass@1高于非结构化检索基线。

C. 相关示例的检索

人们可以通过隔离相关上下文来减少提示中的总上下文。检索增强生成使用嵌入相似性来检索外部知识，并使模型的生成以该检索到的上下文为条件[26]。许多系统通过将示例候选索引到向量存储中并选择余弦相似度下的top-k最近邻来实现密集检索[31]-[33]。一些部署使用最小相似度阈值来过滤示例候选，要么单独使用，要么与top-k结合使用，以减少低相关性上下文[34]、[35]。对于编码专家经验并记录可执行多步骤工具使用（如调用顺序、参数模式和中间状态交接）的过程性示例，固定的top-k检索会产生覆盖范围与噪声之间的权衡[18]、[36]。这个问题对于工具调用和执行智能体尤为严重，因为检索到的内容必须编码可执行结构——调用顺序、参数模式和先决条件依赖关系——而不是叙述性描述[16]。

D. 工具执行的运行时监督

另一个关键挑战出现在工具在残留或不正确状态下执行而不引发异常时，导致静默失败。这些主要是由于违反了工具依赖关系，导致它们在过时或未初始化的环境状态上操作。作为一种轻量级缓解措施，长上下文提示技术可以纳入依赖关系或违反依赖关系和状态约束的对抗性示例，以引导工具选择和执行[37]、[38]，但采样仍然是概率性的，无法保证依赖关系或约束的满足。事后自我修正方法根据检测到的失败修订动作[39]，但当工具在过时或未初始化的对象上无错误执行时，它们无法防止静默错误。一些系统通过在智能体编排器和执行器之间插入确定性的防护栏来实现运行时监督，在每个调用前检查前置条件，并在违反时返回有针对性的反馈。例如，[40]将每个工具表示为一个状态转换规则，带有明确的进入状态要求和预期的状态更新，然后使用符号检查来验证转换是否允许。此外，[41]模拟候选动作以预测执行前的违规行为。最后，[42]定义了事件触发的安全策略，用于门控高风险动作（例如，要求用户确认未经验证的转账接收者）。然而，它们检查的是单步安全性，并没有针对示例检索下的长期工作流正确性。

III. 预备知识

A. 智能体AI系统

一个智能体AI系统将智能体和LLM与一个有状态环境在一个闭环中耦合起来。我们将智能体建模为一个编排器，它将交互历史与相关上下文映射到下一个工具调用动作；将环境建模为一个执行器，它执行调用并返回一个观察。在一个闭环内，在每次迭代k，智能体编排并发出一个工具调用动作a_k，工具在有状态环境的当前状态下执行，以产生下一个状态和观察。

$$\text{Agent} \xrightarrow{a_k} \text{Environment} \xrightarrow{o_k} \text{Agent} \xrightarrow{a_{k+1}} \dots$$

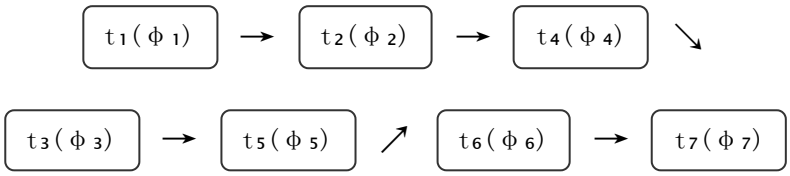
B. 状态转换与工具分类

环境在迭代k时维护一个状态$s_k \in \mathcal{S}$。每个工具$t \in \mathcal{T}$定义一个状态-观察映射$F_t : \mathcal{S} \times \Phi_t \to \mathcal{S} \times \mathcal{O}$，其中$\Phi_t$是工具$t$的参数空间，$\mathcal{O}$是返回给智能体的观察空间。我们根据工具是否修改环境状态对其进行分类：(i) 写工具更新存储在注册表或仿真器中的持久对象，因此$s_{k+1} \neq s_k$；(ii) 读工具查询当前对象并返回一个观察，同时保持环境状态$s_{k+1} = s_k$。一个读工具只有在所需的写工具已经构建了所需状态后才变得有效，这强制了一个由工作流捕获的执行顺序，该工作流用有向无环图建模。如果一个工具改变了任何持久环境对象，我们将其标记为写工具；否则为读工具，并通过状态打印测试验证这一点。

C. 工作流作为有向无环图

智能体通过推荐一组受状态依赖约束的工具调用来响应查询q_u。我们将此工作流形式化为一个有向无环图$\mathcal{G}_w = (V_w, E_w)$。顶点集$V_w = \{v_1, \dots, v_m\}$表示不同的工具调用，其中每个$v_i = t_i(\phi_i)$由一个工具$t_i \in \mathcal{T}$及其参数$\phi_i$组成。边集$E_w \subset V_w \times V_w$编码执行约束：有向边$(v_i, v_j) \in E_w$意味着严格的顺序，确

保v_i在v_j开始之前完成。例如，具有相同父节点且彼此之间没有依赖路径的两个调用可以按任意顺序执行（例如图1中的$t_4(\phi_4)$和$t_5(\phi_5)$）。此外，一个工具只能在其所有传入依赖完成后才能执行（例如$t_6(\phi_6)$依赖于$t_4(\phi_4)$和$t_5(\phi_5)$）。无环性排除了循环先决条件，并保证了拓扑顺序的存在。



node $t_i(\phi_i) \in V_w$: tool invocation
edge $(t_i(\phi_i), t_j(\phi_j)) \in E_w$: ordering-based dependencies

Fig. 1. Workflow as a directed acyclic graph (DAG). Nodes denote tool invocations; directed edges encode ordering-based dependencies.

D. 带注释的查询-工作流对示例

电力系统专家（包括研究人员和工程师）通过遵循既定的工具序列来解决复杂的配电网分析查询。我们将这些经验编码为带注释的查询-工作流对示例。每条记录存储$(q^{(i)}, w^{(i)})$，其中$q^{(i)}$是一个自然语言查询，$w^{(i)}$是在有状态环境中产生所需输出的已执行工作流。

IV. 一个智能体AI系统：PowerDAG

非正式问题：

- 给定一个未见过的自然语言配电网分析查询q_u，一个工具集$\mathcal{T}$及相关数据管道，一个有状态环境，以及专家经验的带注释示例$\mathcal{W}_{av}$，
- 目标是返回查询的准确分析输出以及生成这些输出的可执行工作流w。

PowerDAG是一个智能体AI系统，用于自动化复杂的配电网分析。我们介绍了PowerDAG架构的两个创新点：(i) 自适应示例检索器和 (ii) 即时先决条件监督，这使得PowerDAG在正确性和性能方面超越了现有技术水平。

A. PowerDAG系统架构

令$\mathcal{T}$表示工具集，Σ表示对应的工具模式集。对于每个工具$t \in \mathcal{T}$，我们定义一个模式$\sigma_t = \langle n_t, d_t, \Pi_t \rangle$，其中$n_t$是工具名称，$d_t$是简短描述，$\Pi_t$是参数签名，即输入参数及其数据类型的列表。调用工具t时，会为这些参数使用具体值ϕ_t。

$$\Sigma \backslash \text{coloneqq} \{\sigma_t\}_{t \in \mathcal{T}} \quad (1)$$

我们还维护一个由专家生成的带注释查询-工作流对示例的存档$\mathcal{W}_{av}$。对于查询q_u，一个自适应检索器选择一个子集$\mathcal{W}_{sub} \subset \mathcal{W}_{av}$，并将$\mathcal{W}_{sub}$与$q_u$、交互历史$H$和$\Sigma$一起附加到提示中。然后系统运行智能体与环境之间的闭环交互，如图2所示。在迭代k，智能体编排一个动作：

$$a_k \sim \pi_\theta(\cdot | q_u, \mathcal{W}_{sub}, H_k, \Sigma) \quad (2)$$

智能体要么终止并返回一个最终文本响应，要么返回一个工具调用动作$a_k = \langle n_k, \phi_k \rangle$。

在PowerDAG中，我们在智能体与环境之间放置了一个确定性的即时监督器（见图2），因此监督器在执行前根据先决条件规则$\mathcal{C}$检查提议的工具调用动作。如果违反规则，监督器阻止调用并将建议观察

$o_k = \alpha(n_k)$返回给智能体；不执行任何工具，环境状态保持不变。否则，执行器在环境中执行$t_k(\phi_k)$以产生新状态和观察(s_{k+1}, o_k)，之后历史更新为$H_{k+1} = H_k \oplus (a_k, o_k)$。接下来，我们描述PowerDAG的关键创新：一个从$\mathcal{W}_{\text{av}}$中选择子集$\mathcal{W}_{\text{sub}}$用于查询条件化的自适应检索算法，以及一个在工具调用执行前验证前置条件并返回建议反馈的即时监督算法。

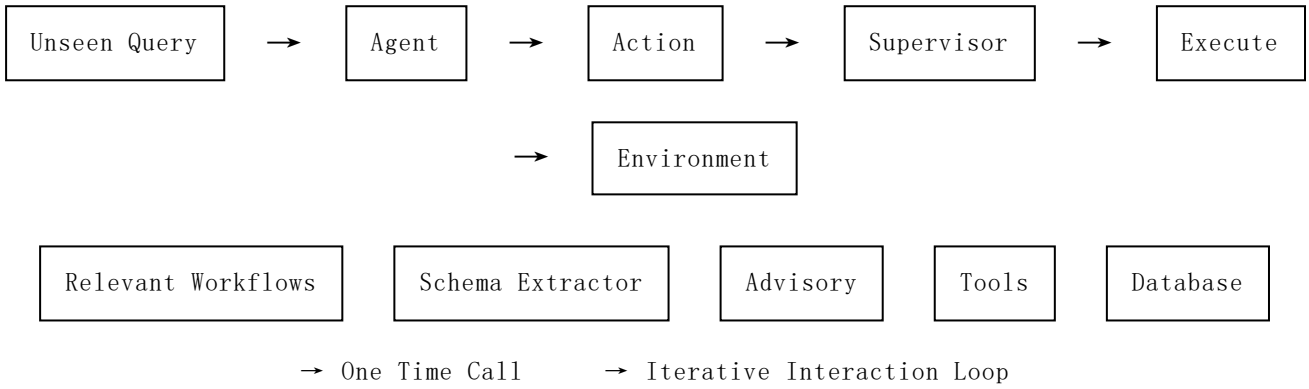


Fig. 2. PowerDAG execution architecture. Initialization: the schema extractor builds Σ from the tool set, and a retrieval module selects relevant W_sub from W_av for the unseen query q_u. Iterative loop: the agent proposes an action, the supervisor either blocks and returns an advisory or executes the action. If executed, an observation returns. The loop stops when the agent gives a textual final response.

B. 带注释示例的自适应检索

我们的目标是开发一种自适应方法，从专家存档$\mathcal{W}_{\text{av}}$中选择带注释的示例$\mathcal{W}_{\text{sub}}$，以克服选择固定超参数k的top-k示例的缺点[16]。为此，我们开发了 (i) 根据排序后的相似度分布计算的自适应断点，以及 (ii) 一个第二阶段过滤器，仅保留其工作流与q_u结构一致的示例。

专家存档为$\mathcal{W}_{\text{av}} = \{(q_i, w_i)\}_{i=1}^{N}$，其中每条记录将存储的查询$q_i$与专家注释的工作流$w_i$配对。上下文选择器通过三个步骤形成文本相似度排序：它将未见查询q_u和每个存储查询q_i编码为嵌入$\psi(\cdot)$，并计算嵌入之间的余弦相似度。然后它对分数进行排序，并按降序相似度分数重新索引$\mathcal{W}_{\text{av}}$中的带注释示例，$i \to n$，使得：

$$\begin{aligned} &\text{Sim}_i = \cos(\psi(q_u), \psi(q_i)), \quad i = 1, \ldots, N \\ &\text{Sim}_1 \geq \text{Sim}_2 \geq \cdots \geq \text{Sim}_N \end{aligned} \tag{4}$$

上下文选择器将$\{\text{Sim}_i\}_{i=1}^{N}$视为一个降序排列的离散相关性分布（即最相关的示例出现在第一个，最不相关的出现在最后一个），并通过将两段最小二乘模型拟合到序列并选择最小化加权拟合误差的断点来选择自适应断点（由有序集中的索引i_{cutoff}给出）：

$$i_{\text{cutoff}} = \arg\min_{2 \leq m \leq N-2} \left[\frac{m}{N} \Phi(1, m) + \frac{N-m}{N} \Phi(m+1, N) \right] \tag{5}$$

这里$\Phi(a, b)$表示在$n = a, \ldots, b$上(n, Sim_n)的最小二乘仿射拟合的RMSE。附录C给出了该目标的完整推导。这产生候选集：

$$\mathcal{W}_{\text{and}} = \{(q_n, w_n)\}_{n=1}^{i_{\text{cutoff}}} \tag{6}$$

仅靠文本相似度排序是不够的，因为它可能检索到查询文本与q_u相似但其相关工作流与q_u所暗示的工具结构不匹配的候选。基于这种不匹配示例的条件化会误导智能体并降低正确性。在检索时，q_u的正确工作流是未知的，因此我们无法直接验证过程性对齐。因此，我们应用一个基于LLM的过滤器，使用查询文本和经过专家验证的工作流轨迹对每个候选进行评分，并仅保留那些与q_u过程一致的候选。LLM仅作

为一个门控机制作用于一个小型候选集；它在现有的专家轨迹中进行选择，并不合成新的工具序列。这种有边界的判别任务比长期工作流生成更简单。过滤器在初始化时运行一次，并返回最终子集$\mathcal{W}_{\text{sub}}$：

$$J = \Omega\big(\pi_{\text{filter}}^{\text{ffor}}(\cdot|q_u, \mathcal{W}_{\text{and}})\big), \quad \mathcal{W}_{\text{sub}} \coloneqq \{(q_j, w_j) : j \in J\} \quad (7)$$

其中Ω是一个确定性解析器，将LLM输出转换为索引集J。附录D列出了确切的过滤提示。图3说明了用于构建$\mathcal{W}_{\text{sub}}$的两阶段选择。

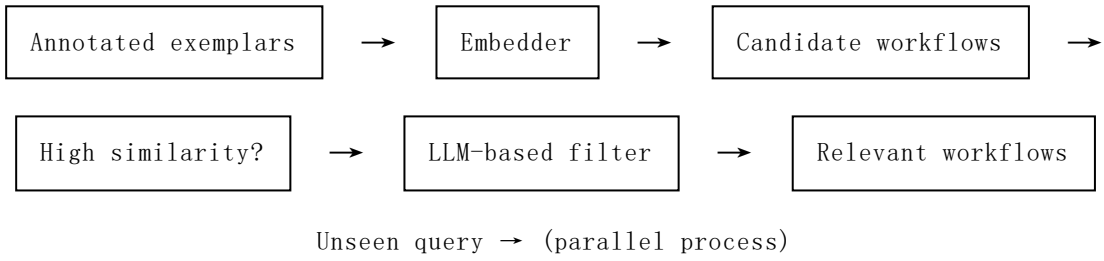


Fig. 3. Two-stage exemplar selection. The system embeds q_u and $\{q_i\}_{i=1}^{N}$, ranks by cosine similarity, selects an adaptive cutoff i_{cutoff}, and applies an LLM-based workflow filter to produce the final subset $\mathcal{W}_{\text{sub}}$.

C. 即时监督

即使能够访问相关的示例$\mathcal{W}_{\text{sub}}$，智能体也可能提出一个工具调用，其所需的先决条件（或依赖）写工具尚未执行，因此该调用作用于过时的状态，并可能错误地覆盖后续步骤所需的存储对象，从而产生复合错误问题。我们通过插入一个确定性的监督器来解决这种失败模式，该监督器使用一个显式规则库$\mathcal{C}$在每个工具执行前检查先决条件，该规则库编码了一个保守的“必须先运行”依赖子集。我们根据以下内容构建$\mathcal{C}$：(i) 工具文档和实现检查，确定每个工具读取和写入哪些状态对象，以及 (ii) 从带注释的查询-工作流对示例$\mathcal{W}_{\text{av}}$中学习重复出现的先决条件模式。

令$\mathcal{N}$表示工具名称集，$\text{dom}(\mathcal{C}) \subseteq \mathcal{N}$表示由先决条件规则覆盖的子集。对于每个$n \in \text{dom}(\mathcal{C})$，该规则指定一个先决条件集$\mathcal{N}^{\text{req}}(n) \subseteq \mathcal{N}_{\text{write}}$和一个建议消息$\alpha(n)$。如果$n_k \notin \text{dom}(\mathcal{C})$，监督器不应用先决条件检查并允许执行。

形式上，给定一个提议的工具调用$a_k = \langle n_k, \phi_k \rangle$，如果执行历史未能满足$n_k$的先决条件集，监督器会标记违规。在这种情况下，监督器拦截该动作，阻止执行，并返回一个明确列出缺失写工具的建议。我们通过附录E中的提示实现此建议，该提示插入违规通知和被阻止的工具调用动作名称。相反，如果未检测到违规，监督器授权该工具执行。

1) 先决条件约束规范

基于读/写工具分类，我们将先决条件知识编码为一个按工具名称索引的显式规则库$\mathcal{C}$。令$\mathcal{N}$表示工具名称全集，$\mathcal{N}_{\text{write}} \subseteq \mathcal{N}$表示写工具集。对于监督器覆盖的每个工具名称$n \in \mathcal{N}$，相应的规则指定 (i) 必须在之前执行的先决条件写工具，以及 (ii) 当缺少先决条件时返回的固定建议消息。我们将此规则表示为

$$c(n) = \langle n, \mathcal{N}^{\text{req}}(n), \alpha(n) \rangle \quad (8)$$

其中$\mathcal{N}^{\text{req}}(n) \subseteq \mathcal{N}_{\text{write}}$是所需的先决条件集，$\alpha(n)$是当$\mathcal{N}^{\text{req}}(n)$未满足时返回的建议消息。如果某个工具名称$n$在$\mathcal{C}$中没有条目，监督器对该工具的调用不应用先决条件检查。

2) 先决条件检查和一次性建议

给定约束集$\mathcal{C}$，监督器在运行时强制执行先决条件，在检测到违规时发出一次性建议。具体来说，在任何步骤k，当智能体提出一个工具调用动作$a_k = \langle n_k, \phi_k \rangle$时，监督器根据已执行工具的历史$\mathcal{N}_k^{\text{exec}} \subseteq \mathcal{N}$（在步骤$0, \ldots, k-1$中累积）对其进行评估。如果$n_k$受$\mathcal{C}$中规则的约束，监督器通过检查$\mathcal{N}^{\text{req}}(n_k) \subseteq \mathcal{N}_k^{\text{exec}}$来验证其依赖关系是否满足。当此包含条件失败时，通过一个指示函数标记违规：

$$\nu_k \backslash \text{coloneqq} \mathbf{1}[\mathcal{N}^{\text{req}}(n_k) \notin \mathcal{N}_k^{\text{exec}}] \quad (9)$$

等价地，$\nu_k = 1$当且仅当该动作要求至少一个尚未执行的先决条件工具。如果$\nu_k = 1$，监督器阻止提议的调用并返回建议观察$o_k = \alpha(n_k)$。为防止无限循环，我们强制执行一次性策略，允许随后重复调用。否则，监督器允许执行。此行为定义了返回观察o_k的确定性更新规则。

$$o_k = \begin{cases} \alpha(n_k) & \text{if} \nu_k = 1 \\ \mathcal{E}(t_k(\phi_k)) & \text{otherwise} \end{cases} \quad (10)$$

算法1描述了PowerDAG的端到端执行。

算法1 PowerDAG算法（详细步骤见译文原文）

V. 实验设置

本节定义了用于衡量PowerDAG正确性和效率的查询、基线、模型和评分协议。

A. 查询与研究范围

我们在30个由专家策划的配电网查询（附录A）上评估PowerDAG。查询集涵盖四类分析：(i) 纯数据查询，(ii) 不平衡三相潮流[43]，(iii) 具有$\ell_1/\ell_2/\ell_\infty$公式的动态承载容量[44]-[47]，以及 (iv) 具有ℓ_1和ℓ_2目标变体的三相不可行性分析[48]、[49]。查询的工作流长度从≤ 3个工具调用到> 10个工具调用不等。

B. 带注释查询-工作流对示例的整理

我们整理了50个带注释的查询-工作流对示例用于上下文学习。每个对包括一个专家撰写的查询以及一个经过完全验证的工作流。所有50个带注释的查询-工作流对示例均与第V-A节中的30个评估查询不同。50个带注释的查询-工作流对示例中有10个涵盖了与30个未见查询相同的四类分析。为了评估PowerDAG的检索准确性，其余40个带注释的查询-工作流对示例代表了与30个未见查询中的四类分析不同的分析。代码和数据可在 https://github.com/PowerDAG/PowerDAG 获取。

C. 工具集、数据接口和智能体系统

表I总结了我们评估中比较的系统及其可用功能。为确保受控比较，我们在所有智能体系统中保持环境、数据管道和工具集固定。完整工具集包含82个工具；30个评估查询仅需要21个工具，涵盖数据加载、求解器运行、约束检查、绘图和导出。我们在评估时有意添加了61个工具，以测试智能体是否能在扩大的动作空间中选择正确的工具。我们在真实世界的配电馈线上运行所有实验（由于双盲评审，此版本中不具名）。数据管道包括AMI数据库、GeoJSON GIS文件、时间序列光伏发电数据以及对应于同一真实世界公用事业的GridLAB-D格式的网络数据。

表**I** 基线系统与启用组件比较

组件	**Vanilla ReAct [13]**	**LangChain [50]**	**CrewAI [51]**	**PowerChain [16]**	**PowerDAG (ours)**
工具编排	✓	✓	✓	✓	✓
动作-观察循环	✓	✓	✓	✓	✓

组件	Vanilla ReAct [13]	LangChain [50]	CrewAI [51]	PowerChain [16]	PowerDAG (ours)
ICL 示例	×	×	×	✓	✓
检索	×	×	×	✓	✓
自适应检索	×	×	×	×	✓
JIT 监督器	×	×	×	×	✓

D. LLM 和计算平台

我们通过OpenAI API评估了两个专有LLM：GPT-4o-MINI [52] 和 GPT-5.2 [53]。我们还评估了四个开源权重LLM：Qwen3-14B [55]、Qwen3-32B [56]、GPT-oss-20B [57] 和 GPT-oss-120B [57]。我们在一个具有2块NVIDIA H100 SXM GPU（各80 GB）的节点上使用vLLM服务器[58]提供开源权重模型服务[59]。对于示例选择，我们使用OpenAI文本嵌入器TEXT-EMBEDDING-3-LARGE [60]嵌入查询和存档的示例文本。

E. 评估与指标

我们使用pass@k和精确率（Precision）评估PowerDAG和基线智能体系统在每个未见评估查询上生成的工作流的正确性。对于每个查询和方法，智能体生成$k \in \{1, 3\}$个独立工作流，因为LLM解码是非确定性的，因此工具调用轨迹可能因试验而异[61]。Pass@k然后衡量这k个工作流中是否至少有一个在相同环境中执行时产生与专家基准工作流相同的数值结果；我们报告pass@1和pass@3。精确率衡量智能体执行的工具调用轨迹是否等同于专家基准工作流。我们还报告一个效率指标：每个pass@1的令牌数（Tokens per pass@1），它衡量每次成功运行的平均令牌成本。附录F提供了完整的指标定义和方程，包括我们为计算这些指标而编写的评估器的描述。

VI. 结果

本节报告PowerDAG在未见配电网分析查询上的基准测试结果。表II将PowerDAG与四个基线（CrewAI、LangChain、Vanilla ReAct和PowerChain [16]）在六个LLM上进行了比较。PowerDAG在所有模型上实现了最高的Pass@1和Pass@3率。它每次成功运行使用的令牌也更少。改进在较小的模型（GPT-4o-mini、GPT-OSS-20B、Qwen3-14B）上最大，其他智能体在这些模型上无法完成多步骤工作流。我们从四个部分分析这些结果：基线比较、模型缩放行为、效率分析和消融研究。

表**II** 智能体系统在各指标上的比较

Method	GPT-4o-mini				GPT-5.2			
	P@1	P@3	Pr	Tk/P@1(K)	P@1	P@3	Pr	Tk/P@1(K)
CrewAI	41.11	43.33	28.89	920.17	34.44	43.33	17.78	719.17
LangChain	43.33	46.67	4.44	111.40	90.00	93.33	71.11	58.61
Vanilla ReAct	41.11	46.67	0.00	16.28	87.78	93.33	81.11	7.72
PowerChain	76.67	83.33	60.00	11.04	96.67	96.67	96.67	11.59
PowerDAG	**94.44**	**96.67**	**77.78**	**8.98**	**100.00**	**100.00**	**100.00**	**8.64**

（其余模型对比表格因篇幅不再重复，完整数据请参考原译文）

Language Model	CrewAI	Vanilla ReAct	LangChain	PowerDAG	PowerChain

Language Model	CrewAI	Vanilla ReAct	LangChain	PowerDAG	PowerChain
GPT-4o-mini	1	20	0	46	76
Qwen3-14B	2	0	0	64	85
GPT-Oss-120B	1	30	0	66	85
GPT-Oss-20B	5	0	0	74	90
Qwen3-32B	1	40	0	70	90
GPT-5.2	6	0	0	72	98

Fig. 4. Combined performance score (Pass@1 × Precision) across models. This metric captures both first-attempt success and workflow correctness. PowerDAG achieves the highest scores on all six models, with near-perfect performance on four of them. The gap between PowerDAG and baselines is largest on smaller models.

A. 与基线的性能比较

表II显示，PowerDAG在每个模型上都优于所有基线。在GPT-5.2上，PowerDAG取得了满分：100% Pass@1、100% Pass@3和100%精确率。PowerChain在所有三个指标上达到96.67%。LangChain作为一个开源智能体工作流编排器，达到了90%的Pass@1，而Vanilla ReAct和CrewAI表现更差。在GPT-OSS-120B上，PowerDAG保持94.44%的Pass@1和90.0%的精确率。PowerChain达到84.44%的Pass@1，Vanilla ReAct为86.67%，尽管Vanilla ReAct的精确率较低（75.56%）。LangChain和CrewAI显示出进一步的退化。

B. 在较小模型上的鲁棒性

PowerDAG在较小模型上保持高性能，在GPT-4o-mini、GPT-OSS-20B、Qwen3-14B和Qwen3-32B上达到94%-97%的Pass@1。标准基线表现不佳：Vanilla ReAct降至41%-60%的Pass@1，而CrewAI和LangChain降至50%以下。与PowerChain相比，PowerDAG将Pass@1提高了10-18个百分点，精确率提高了9-18个百分点。在GPT-4o-mini上增益最大，PowerDAG达到94.44%的Pass@1，而PowerChain达到76.67%。这些结果表明，PowerDAG使更小、更具成本效益的模型能够用于复杂的电网分析任务。

C. 效率与令牌经济性

在大多数设置中，PowerDAG每次成功运行使用的令牌少于所有基线（表II）。在GPT-4o-mini上，PowerDAG每个Pass@1需要9.0k令牌，而Vanilla ReAct需要16.3k。LangChain和CrewAI的效率低得多，分别为111.4k和920.2k。两个因素解释了这些增益。首先，PowerDAG犯的错误更少。更高的Pass@1意味着更少的重试和失败轨迹。虽然PowerDAG通过包含检索到的示例增加了上下文令牌，但这种开销是值得的：PowerDAG在第一次尝试时就能正确完成更多工作流。其次，PowerDAG避免了提示膨胀。LangChain和CrewAI在每次调用中都包含大量的工具描述和基于角色的上下文。这种开销推送了太多信息，模型失去了注意力[18]。

D. 所提出扩展的端到端影响

我们将PowerDAG与PowerChain [16]进行比较，以量化本文引入的两个扩展（自适应检索和即时监督）的端到端效果。表II报告了两个系统在评估模型上的Pass@1、Pass@3和精确率。最大的增益出现在较小模型上。例如，在GPT-4o-mini上，Pass@1从76.7%增加到94.4%，精确率从60.0%增加到77.8%。在Qwen3-14B上，Pass@1从82.2%增加到95.6%，精确率从76.7%增加到88.9%。PowerDAG在四个模型上达

到了100%的Pass@3，而PowerChain未能达到。结果与自适应检索和即时监督的预期作用一致，但这种比较并未隔离每个扩展的边际贡献。

VII. 结论

PowerDAG通过在有状态环境中编排和执行工具调用序列来自动化配电网分析。PowerDAG通过以下关键贡献提高了可靠性：

- 正确性：自适应上下文检索与即时监督帮助大多数模型达到接近100%的准确率。
- 模型效率：更高的首次尝试成功率减少了对迭代修正的需求，显著降低了总令牌消耗和运行时间。
- 普及化：在开源模型上的高精度表明PowerDAG可以避免使用昂贵的商业模型。

附录

AI使用披露：我们使用生成式AI来修改语法和改进写作。我们审阅了每一条建议，并对最终内容负责。

附录**A** 基准任务集

简单
1. 佛蒙特州Rochester馈线中有多少个电容器？
10. 你能将South Hero馈线中所有电压为120伏的节点导出到一个txt文件中吗？

中等
1. 在不运行任何分析的情况下，绘制2025年3月18日上午11:00 Glover配电网输入数据中的母线电压幅值。
20. 对于2025年3月21日下午6:00的Glover网络，运行L1电流不可行性分析。设置收敛容差为1e-6。报告不可行电流最高的前5条母线。然后，在此分析后绘制电压幅值。

困难
1. 加载2025年3月19日15:00佛蒙特州Rochester馈线的网络文件。运行动态承载容量。假设该日期和时间的太阳辐照度数据。选择稀疏削减策略。在所有节点上强制执行0.95至1.05的电压限制。将变压器流量限制在额定值的110%。解决问题后，将削减功率绘制为馈线拓扑图，每个母线按其削减功率值着色，边根据网络连接性绘制。
30. 对Stowe馈线进行L2范数电流不可行性研究，代表2025年3月23日10:00的条件。在此分析中，对所有母线施加0.94-1.06 p.u.的全局电压，但ID为10、25和36的母线除外。这些母线的电压应在0.98至1.02 p.u.之间。此外，强制执行全局变压器负载限制为额定值的115%。使用IPOPT求解模型，容差为1e-6，最大迭代次数为1000。解决问题后，在地图上生成一个可视化每条母线不可行电流的图，以及另一个在地图上显示母线电压的图。

附录**B** 带注释的已验证工作流-任务对

1. 加载{location}馈线模型，并报告母线、线路和变压器的数量作为快速完整性检查。
10. 对于{location}馈线在{YYYY-MM-DD} {HH:MM}，运行带有变压器限制110%的L2电流不可行性分析，应用0.90-1.10 p.u.的全局电压界限和针对第5、10和30条母线的0.95-1.05 p.u.自定义界限。然后绘制母线电压和不可行电流。

完整基准和示例见 https://github.com/PowerDAG/PowerDAG

附录**C** 自适应断点目标的推导

$$I_L(m)\coloneqq\{1,\ldots,m\},\quad I_R(m)\coloneqq\{m+1,\ldots,N\}$$

$$(\hat{\alpha}_{a:b},\hat{\beta}_{a:b})\coloneqq\arg\min_{\alpha,\beta\in\mathbb{R}}\sum_{n=a}^{b}\left(\mathrm{Sim}_n-(\alpha+\beta n)\right)^2$$

$$\Phi(a,b)=\sqrt{\frac{1}{b-a+1}\sum_{n=a}^{b}\Big(\mathrm{Sim}_n-(\hat{\alpha}_{a:b}+\hat{\beta}_{a:b}n)\Big)^2}$$

$$\mathcal{E}(m)=\frac{m}{N}\Phi(1,m)+\frac{N-m}{N}\Phi(m+1,N)$$

附录**D** 工作流过滤提示

用户查询：{query}
候选（JSON）：{candidates}
仅返回一个要保留的索引的JSON列表（无重复）。示例：[0, 2, 5]

附录**E** 监督建议提示

需要决策：{violation_msg}
1. 如果该查询需要此先决条件：首先调用建议的工具
2. 如果不需要：现在重试你对{tool_name}的原始调用
进行一次工具调用。不要用文本回复。

附录**F** 评估指标

$$\text{Pass@k}=1-\frac{\binom{n-s}{k}}{\binom{n}{k}}\quad \Pr=\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}[\hat{w}_i\equiv w_e]\quad \text{Token/Pass@1}=\frac{\sum T_i}{n\cdot\text{Pass@1}}$$

本文献翻译自 PowerDAG 论文，仅供学术交流使用。内嵌图片根据您提供的文本重新绘制，数学公式由 MathJax 渲染。